

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	10% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	J KARTHIK	Reg. No.:	192472136
Section / Batch:	CSE – AI / 2024	Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze the given questions thoroughly before answering.
- Compare multiple aspects such as methods, results, advantages, and limitations clearly.
- Critically evaluate the strengths, weaknesses, and implications with proper justification.
- Present meaningful insights, interpretations, and well-supported conclusions from the comparisons.

Question Type	Focus Area	Questions
Comparative Analysis	Comparative analysis of Dynamic Programming and Greedy algorithms for optimization problems.	Q1

SECTION A – Comparative Analysis

Code of Conduct

/ J KARTHIK / 192472136 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: J KARTHIK

Question 1: Knapsack Comparison (0/1 DP vs Fractional Greedy)

Problem Statement

Items have weights [10, 20, 30] and values [60, 100, 120]. Knapsack capacity = 50. Solve using (a) 0/1 Knapsack via Dynamic Programming and (b) Fractional Knapsack via Greedy. Compare optimality, approach, strengths/limitations, and generate insights on method selection.

Required Parameters

- Items: Item1 (w=10, v=60), Item2 (w=20, v=100), Item3 (w=30, v=120)
- Value-to-weight ratio: Item1 = 6, Item2 = 5, Item3 = 4
- Capacity $W = 50$

Method 1: 0/1 Knapsack (Dynamic Programming)

Since items cannot be broken, DP builds a table $dp[i][c]$ = best value using the first i items with capacity c , using the recurrence:

$$dp[i][c] = dp[i-1][c] \quad \text{if weight}[i] > c$$
$$dp[i][c] = \max(dp[i-1][c], dp[i-1][c - \text{weight}[i]] + \text{value}[i]) \quad \text{otherwise}$$

Evaluating all feasible combinations within capacity 50:

Item1 + Item2 → weight 30, value 160

Item1 + Item3 → weight 40, value 180

Item2 + Item3 → weight 50, value 220 (best feasible, exact fit)

Item1 + Item2 + Item3 → weight 60 (exceeds capacity, infeasible)

Optimal 0/1 result: pick Item2 and Item3 → Total value = 220, weight used = 50.

Method 2: Fractional Knapsack (Greedy)

Greedy sorts items by value/weight ratio in descending order: Item1 (6) > Item2 (5) > Item3 (4), and fills the sack greedily.

Take Item1 fully : weight 10, value 60 | remaining capacity = 40

Take Item2 fully : weight 20, value 100 | remaining capacity = 20

Take 20/30 of Item3 : weight 20, value $120 * (20/30) = 80$

Total value = $60 + 100 + 80 = 240$

Optimal fractional result: Total value = 240, using the full capacity of 50.

Comparison of Solutions

Aspect	0/1 Knapsack (DP)	Fractional Knapsack (Greedy)
Items selected	Item2 + Item3 (whole items only)	Item1 + Item2 + 2/3 of Item3
Optimal value	220	240
Capacity utilised	50 / 50 (exact)	50 / 50 (exact)
Time complexity	$O(n \times W)$	$O(n \log n)$ for the sort
Space complexity	$O(n \times W)$ ($O(W)$ with 1-D array)	$O(1)$ extra
Decision basis	Explores all subsets via subproblem table	Local best ratio at each step

Critical Evaluation

The greedy fractional value (240) is numerically higher than the DP 0/1 value (220), but the two are not directly interchangeable answers — they solve different problems. Greedy is only optimal here because items may be split; if items must be taken whole (the realistic 0/1 case), greedy's ratio-based choice can strand capacity or force a low-value filler item, so it is not guaranteed to reach the true optimum. DP is exhaustive over feasible subsets (via overlapping subproblems and memoised subresults), so it always finds the true 0/1 optimum, at the cost of higher time and space usage. Greedy is fast and simple but its correctness depends on the problem having the greedy-choice property and optimal substructure that holds under fractional relaxation — properties the discrete 0/1 version does not have.

Insights on Method Selection

- Use Greedy (fractional) when items are divisible, e.g., liquids, grains, budget allocation in continuous units — it is fast ($O(n \log n)$) and provably optimal for the fractional variant.
- Use DP (0/1) when items are indivisible, e.g., selecting projects, packing discrete objects — correctness requires exploring combinations, which only DP guarantees.
- The fractional solution value is always an upper bound on the 0/1 solution value for the same input, useful as a quick bound/heuristic before running exact DP.

Question 4: Coin Change Problem (Greedy vs DP)

Problem Statement

Coin denominations = [1, 3, 4], amount = 6. Solve using Greedy and Dynamic Programming. Compare the number of coins obtained, analyze correctness, and explain why greedy fails here.

Required Parameters

- Denominations: {1, 3, 4}
- Target amount: 6
- Objective: minimise the number of coins used to make the exact amount

Method 1: Greedy Approach

Greedy always picks the largest denomination not exceeding the remaining amount:

Remaining = 6 -> pick 4 -> Remaining = 2 | coins used: [4]

Remaining = 2 -> pick 1 -> Remaining = 1 | coins used: [4,1]

Remaining = 1 -> pick 1 -> Remaining = 0 | coins used: [4,1,1]

Greedy result: 3 coins (4 + 1 + 1)

Method 2: Dynamic Programming Approach

Let $\text{minCoins}[a]$ be the minimum coins to form amount a , $\text{minCoins}[0] = 0$, and:

$\text{minCoins}[a] = 1 + \min(\text{minCoins}[a - c])$ for every coin $c \leq a$

Building the table for $a = 0..6$ with coins {1,3,4}:

$\text{minCoins}[0] = 0$

$\text{minCoins}[1] = 1$ (1)

$\text{minCoins}[2] = 2$ (1+1)

$\text{minCoins}[3] = 1$ (3)

$\text{minCoins}[4] = 1$ (4)

$\text{minCoins}[5] = 2$ (4+1) or (3+1+1) -> best is 4+1

$\text{minCoins}[6] = 2$ (3+3) <-- optimal

DP result: 2 coins (3 + 3).

Comparison of Solutions

Aspect	Greedy	Dynamic Programming
Coins used	4, 1, 1	3, 3
Number of coins	3	2 (optimal)
Correctness	Suboptimal for this coin set	Always optimal
Time complexity	$O(\text{amount} / \text{smallest coin})$ approx	$O(\text{amount} \times \text{number of coins})$

Aspect	Greedy	Dynamic Programming
	$O(n)$	
Space complexity	$O(1)$	$O(\text{amount})$

Critical Evaluation

Greedy fails on $\{1,3,4\}$ for amount 6 because this denomination set is non-canonical: the largest-first rule locks in a 4 early, leaving a remainder (2) that cannot be paid as efficiently as if $3+3$ had been chosen instead. Greedy has no mechanism to look ahead or reconsider a choice once made, so a locally best coin can lead to a globally worse total. DP avoids this by evaluating every amount from 0 upward and storing the true optimum for each, so every larger amount is built only from already-optimal smaller solutions (this is the principle of optimality). The trade-off is that DP must compute and store a value for every amount up to the target, which costs more time and memory than greedy's single pass.

Insights on Choosing the Appropriate Method

- Greedy is safe only for canonical coin systems (e.g., standard currency like $\{1,5,10,25\}$) where it is proven to always match the optimum.
- For arbitrary or unknown denominations, DP should be preferred since it guarantees the minimum-coin solution regardless of the coin set's structure.
- This example is a classic proof that the greedy-choice property does not hold universally — it must be verified for the specific input before trusting greedy.

Question 5: Activity Selection (Greedy vs DP)

Problem Statement

Activities have start times [1, 3, 0, 5, 8, 5] and finish times [2, 4, 6, 7, 9, 9]. Solve using Greedy and DP, compare selected activities and efficiency, evaluate the greedy-choice property, and derive insights on scheduling optimization.

Required Parameters

Labelling activities by their (start, finish) pairs:

A1 = (1, 2)

A2 = (3, 4)

A3 = (0, 6)

A4 = (5, 7)

A5 = (8, 9)

A6 = (5, 9)

➤ Objective: select the maximum number of mutually non-overlapping activities

Method 1: Greedy Approach (sort by finish time)

Sorting activities by finish time gives: A1(2), A2(4), A3(6), A4(7), A5(9), A6(9).

Select A1 (1,2) → last finish = 2

A2 (3,4): start 3 $\geq$ 2 → select, last finish = 4

A3 (0,6): start 0 < 4 → reject (overlaps)

A4 (5,7): start 5 $\geq$ 4 → select, last finish = 7

A5 (8,9): start 8 $\geq$ 7 → select, last finish = 9

A6 (5,9): start 5 < 9 → reject (overlaps with A4/A5 window)

Greedy result: {A1, A2, A4, A5} → 4 non-overlapping activities.

Method 2: Dynamic Programming Approach

DP defines $dp[i]$ = size of the largest compatible activity set ending at activity i (sorted by finish time), using:

$dp[i] = 1 + \max(dp[j])$ over all j whose finish time $\leq$ start time of i, else $dp[i] = 1$

Evaluating with the same finish-time order (A1,A2,A3,A4,A5,A6):

$dp[A1] = 1$ (no compatible predecessor)

$dp[A2] = 1 + dp[A1] = 2$ (A1 finishes at 2 $\leq$ A2 start 3)

$dp[A3] = 1$ (A3 starts at 0, no predecessor fits)

$dp[A4] = 1 + dp[A2] = 3$ (A2 finishes at 4 $\leq$ A4 start 5)

$dp[A5] = 1 + dp[A4] = 4$ (A4 finishes at 7 $\leq$ A5 start 8)

$dp[A6] = 1 + dp[A4] = 4$ (A4 finishes at 7 $\leq$ A6 start 5? no $\rightarrow$ check A2: 4 $\leq$ 5 $\rightarrow$ $dp[A6] = 1 + dp[A2] = 3$)

The maximum value in the dp table is 4, achieved by the chain A1 $\rightarrow$ A2 $\rightarrow$ A4 $\rightarrow$ A5, matching the greedy selection exactly.

Comparison of Solutions

Aspect	Greedy	Dynamic Programming
Selected activities	A1, A2, A4, A5	A1, A2, A4, A5
Count	4	4 (same, optimal)
Time complexity	$O(n \log n)$ for sorting	$O(n^2)$ (or $O(n \log n)$ with binary search)
Space complexity	$O(1)$ extra	$O(n)$ for the dp table
Approach	Single greedy pass by finish time	Evaluates compatible-predecessor subproblems

Critical Evaluation

Unlike the coin-change case, greedy succeeds here because unweighted activity selection satisfies both the greedy-choice property and optimal substructure: always choosing the activity that finishes earliest leaves the maximum possible remaining time for future activities, and this local choice never eliminates a better global solution. DP arrives at the identical answer but does strictly more work, since it evaluates every possible compatible predecessor for every activity rather than committing to the earliest-finish choice directly. This makes DP the more general but less efficient tool: it remains correct, but is unnecessary overhead when the greedy-choice property is already provable for the problem.

Insights on Scheduling Optimization

- When each activity has equal weight/value, greedy by earliest finish time is provably optimal and should be preferred for its $O(n \log n)$ efficiency.
- If activities carried different weights/profits (weighted activity selection), the greedy-choice property would break down and DP would become necessary — this mirrors the Job Sequencing vs Knapsack distinction seen elsewhere in this paper.
- DP remains valuable as a verification tool: it confirms greedy's optimality here by independently reaching the same maximum count of 4.

Question 7: String Matching — LCS (Dynamic Programming) vs Greedy Matching

Problem Statement

Given strings $X = \text{"ABCB DAB"}$ and $Y = \text{"BDCAB"}$, compute the Longest Common Subsequence using DP and compare with a greedy matching attempt. Analyze differences in results, evaluate why greedy fails, and generate insights on optimal substructure.

Required Parameters

- $X = \text{"ABCB DAB"}$ (length 7)
- $Y = \text{"BDCAB"}$ (length 5)
- Objective: find the longest subsequence common to both strings (characters in order, not necessarily contiguous)

Method 1: Longest Common Subsequence (Dynamic Programming)

The DP recurrence builds a table $dp[i][j]$ over prefixes of X and Y :

$dp[i][j] = dp[i-1][j-1] + 1$ if $X[i] == Y[j]$

$dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$ otherwise

Completed DP table (rows = X prefixes A,B,C,B,D,A,B; columns = Y prefixes B,D,C,A,B):

	B	D	C	A	B
	0	0	0	0	0
A	0	0	0	1	1
B	0	1	1	1	2
C	0	1	1	2	2
B	0	1	1	2	3
D	0	1	2	2	3
A	0	1	2	3	3
B	0	1	2	3	4

DP result: LCS length = 4. Tracing back through the table recovers the subsequence "BCAB" (B, C, A, B all appear in order in both strings).

Method 2: Naive Greedy Matching

A greedy matcher scans both strings left to right and accepts the first available matching character it meets, without considering later consequences:

X: A B C B D A B

Y: B D C A B

Greedy scan: $X[1]=A$ vs $Y \text{ pointer}(B)$ no match $\rightarrow$ advance X

X[2]=B matches Y[1]=B -> take 'B', advance both pointers

X[3]=C vs Y[2]=D no match -> advance X

X[4]=B vs Y[2]=D no match -> advance X

X[5]=D matches Y[2]=D -> take 'D'? but 'D' does not appear in the true LCS

Continuing greedily typically yields a short, low-value chain such as 'B D' or 'B A B' (length 2-3)

Greedy result: a subsequence of length 2-3 (e.g., "BAB"), shorter than the true LCS of length 4, and in this case it locks in a "D" match that is not part of any longest common subsequence.

Comparison of Results

Aspect	LCS (Dynamic Programming)	Greedy Matching
Result	"BCAB"	"BAB" (or similar short match)
Length	4 (optimal)	2-3 (suboptimal)
Time complexity	$O(m \times n)$	$O(m + n)$
Space complexity	$O(m \times n)$ ($O(\min(m,n))$ optimized)	$O(1)$
Guarantee	Always finds the true longest common subsequence	No optimality guarantee

Critical Evaluation

Greedy matching fails because a locally convenient match (accepting the first available common character) can consume a position that a longer subsequence needed later, with no way to undo that choice. LCS does not have the greedy-choice property: the best decision at a given pair of positions depends on what comes after, not just on what is available right now. DP handles this correctly because it evaluates both possibilities at every position — skip a character in X, skip a character in Y, or match them — and keeps whichever choice leads to the best final result, which is the essence of optimal substructure (the LCS of two full strings is built from the LCS of their prefixes). The cost is quadratic time and space, which becomes significant for very long strings, whereas greedy is linear but unreliable.

Insights on Optimal Substructure

- LCS is a canonical example where greedy heuristics are tempting (simple left-to-right scanning) but incorrect, because the problem lacks a provable greedy-choice property.
- Whenever a problem's optimal solution to the whole depends on optimal solutions to overlapping subproblems whose best choice cannot be determined locally, DP is required rather than greedy.
- For very long sequences where $O(m \times n)$ DP is too costly, approximate/heuristic algorithms are used deliberately as a trade-off — but they are understood to be approximations, unlike greedy being mistaken for an exact method.

Question 10: Huffman Coding vs Fixed-Length Coding

Problem Statement

Given symbol frequencies {A:5, B:9, C:12, D:13, E:16, F:45}, construct Huffman codes and compare with fixed-length encoding. Analyze compression efficiency and code length, and evaluate advantages/limitations.

Required Parameters

- Symbols and frequencies: A=5, B=9, C=12, D=13, E=16, F=45
- Total symbol occurrences = $5+9+12+13+16+45 = 100$
- Number of distinct symbols = 6

Method 1: Fixed-Length Coding

With 6 symbols, the minimum fixed code length is $\text{ceil}(\log_2 6) = 3$ bits per symbol (3 bits allows up to 8 codes).

Total bits = 100 symbols x 3 bits = 300 bits

Method 2: Huffman Coding (Greedy)

Huffman repeatedly merges the two lowest-frequency nodes into a new parent node until one tree remains:

Step 1: merge A(5) + B(9) → N1 = 14

Step 2: merge C(12) + D(13) → N2 = 25

Step 3: merge N1(14) + E(16) → N3 = 30

Step 4: merge N2(25) + N3(30) → N4 = 55

Step 5: merge F(45) + N4(55) → Root = 100

Reading the resulting tree gives the following codeword lengths and codes:

Symbol	Frequency	Huffman Code	Code Length (bits)
F	45	0	1
C	12	100	3
D	13	101	3
E	16	111	3
A	5	1100	4
B	9	1101	4

Total Huffman bits = $(45 \times 1) + (12 \times 3) + (13 \times 3) + (16 \times 3) + (5 \times 4) + (9 \times 4)$

= $45 + 36 + 39 + 48 + 20 + 36$

= 224 bits

Comparison of Compression Efficiency

Aspect	Fixed-Length Coding	Huffman Coding (Greedy)
Code length per symbol	Fixed at 3 bits for every symbol	Variable: 1 to 4 bits, frequency-dependent
Total bits for 100 symbols	300 bits	224 bits
Compression achieved	None (baseline)	$(300-224)/300 = 25.3\%$ smaller
Decoding	Trivial (fixed width, direct indexing)	Requires prefix-free tree traversal
Optimality	Not frequency-aware	Optimal prefix code for given frequencies

Critical Evaluation

Huffman coding outperforms fixed-length coding by 76 bits (25.3% reduction) on this frequency distribution because it exploits the skew in symbol frequencies: the dominant symbol F (45 occurrences) is assigned just 1 bit, while rare symbols A and B receive longer 4-bit codes. This is a direct application of the greedy-choice property for prefix codes: repeatedly combining the two least-frequent nodes provably minimises the total weighted code length, and Huffman's codes are always prefix-free, so no code is a prefix of another, allowing unambiguous decoding without delimiters. The limitation is that Huffman's advantage shrinks as frequencies become more uniform — with perfectly equal frequencies, Huffman degenerates to essentially the same length as fixed-length coding (or very close to it), so the compression gain is data-dependent, not guaranteed. Huffman also requires transmitting or agreeing on the code table/tree, and building and traversing that tree adds implementation complexity that fixed-length coding avoids entirely.

Insights on Real-World Data Compression

- Huffman coding is most beneficial when symbol frequencies are highly skewed (as in natural-language text, where some characters occur far more often than others).
- Fixed-length coding remains preferable when simplicity, constant-time decoding, and random access to encoded data matter more than raw compression ratio.
- Huffman coding is a foundational building block in real compression systems (e.g., DEFLATE/ZIP, JPEG, MP3), typically combined with other transforms, confirming that greedy frequency-based tree construction scales well to practical, real-world compression pipelines.

Overall Summary: DP vs Greedy Across the Five Questions

Question	Problem	Greedy Result	DP Result	Greedy Optimal?
Q1	0/1 vs Fractional Knapsack	240 (fractional)	220 (0/1 exact)	Only for the fractional variant
Q4	Coin Change [1,3,4], amount 6	3 coins	2 coins	No
Q5	Activity Selection	4 activities	4 activities	Yes
Q7	LCS of two strings	Length 2-3 (suboptimal)	Length 4 (optimal)	No
Q10	Huffman vs Fixed-length coding	224 bits (Huffman is greedy)	N/A (Huffman itself is greedy; DP not required)	Yes, provably optimal prefix code

Synthesized Insight

Across all five comparisons, greedy succeeds only when the problem provably satisfies both the greedy-choice property and optimal substructure without conflicting future constraints — as seen in unweighted Activity Selection and in Huffman's frequency-driven prefix coding. Greedy fails whenever an early locally-optimal choice can block a better global outcome, as seen in 0/1 Knapsack, non-canonical Coin Change, and LCS. Dynamic Programming is the safe general-purpose tool because it exhaustively considers overlapping subproblems and combines their optimal solutions, guaranteeing correctness at the cost of higher time and space complexity. The practical takeaway for method selection is to first check whether a problem's greedy-choice property can be proven; if it can, greedy should be preferred for its efficiency, and DP should be reserved for problems where such a proof does not hold or where the problem structure (e.g., discrete/indivisible choices) inherently breaks the greedy assumption.

RUBRICS FOR CALCULATION

Comparative Analysis					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Selection of Studies/Parameters	20%	Selects highly relevant and diverse studies with well-defined comparison parameters	Selects relevant studies with minor gaps in parameter definition	Limited or partially relevant selection	Poor or inappropriate selection of studies
Comparison & Differentiation	25%	Clearly compares multiple aspects (methods, results, limitations) with strong differentiation	Good comparison with minor gaps	Basic comparison with limited depth	No clear comparison; mostly descriptive
Critical Evaluation	25%	Critically evaluates strengths, weaknesses, and implications with strong justification	Provides evaluation with some justification	Limited evaluation; lacks depth	No critical evaluation provided
Synthesis & Insight Generation	20%	Generates meaningful insights and conclusions from comparisons	Some insights derived from comparison	Limited or unclear insights	No insights generated
Clarity	10%	Information is clearly presented (tables/figures) with logical structure	Generally clear with minor issues	Presentation lacks clarity or structure	Poor presentation and difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Selection of Studies/Parameters	20%				
Comparison & Differentiation	25%				
Critical Evaluation	25%				
Synthesis & Insight Generation	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

